DECORATOR (Decorador)

Estructural de Objetos

PROPÓSITO

Asigna responsabilidades adicionales a un objeto dinámicamente, proporcionando una alternativa flexible a la herencia para extender la funcionalidad.

TAMBIÉN CONOCIDO COMO

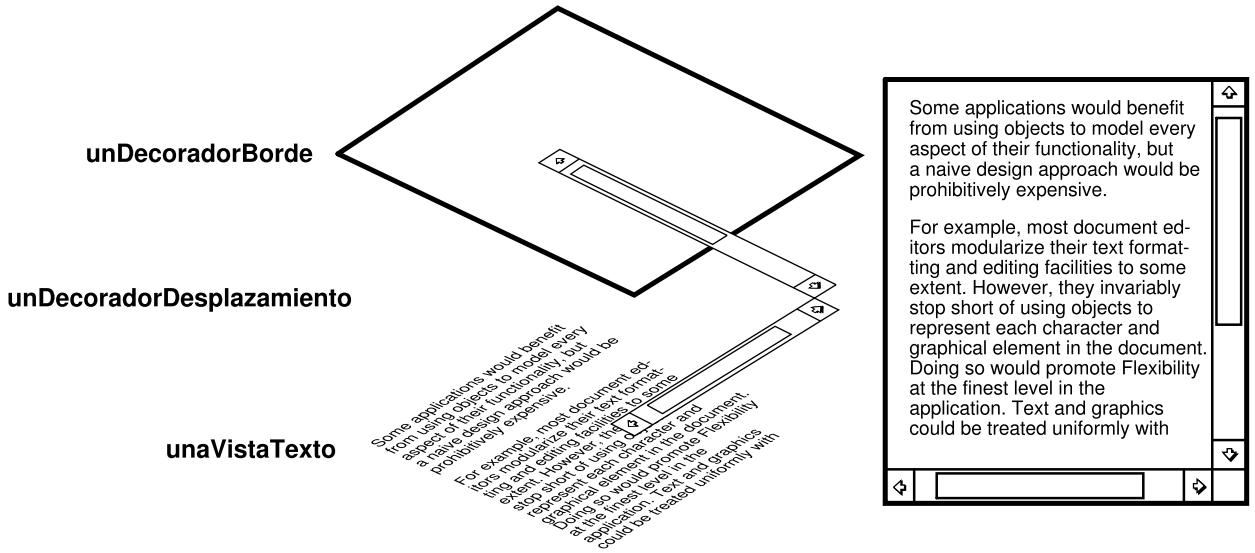
Wrapper (Envoltorio)

MOTIVACIÓN

A veces queremos añadir responsabilidades a objetos individuales en vez de a toda una clase. Por ejemplo, un toolkit de interfaces de usuario debería permitir añadir propiedades (como bordes) o comportamientos (como capacidad de desplazamiento) a cualquier componente de la interfaz de usuario.

Un modo de añadir responsabilidades es a través de la herencia. Heredar un borde de otra clase pondría un borde alrededor de todas las instancias de la subclase. Sin embargo, esto es inflexible, ya que la elección del borde se hace estáticamente. Un cliente no puede controlar cómo y cuándo decorar el componente con un borde.

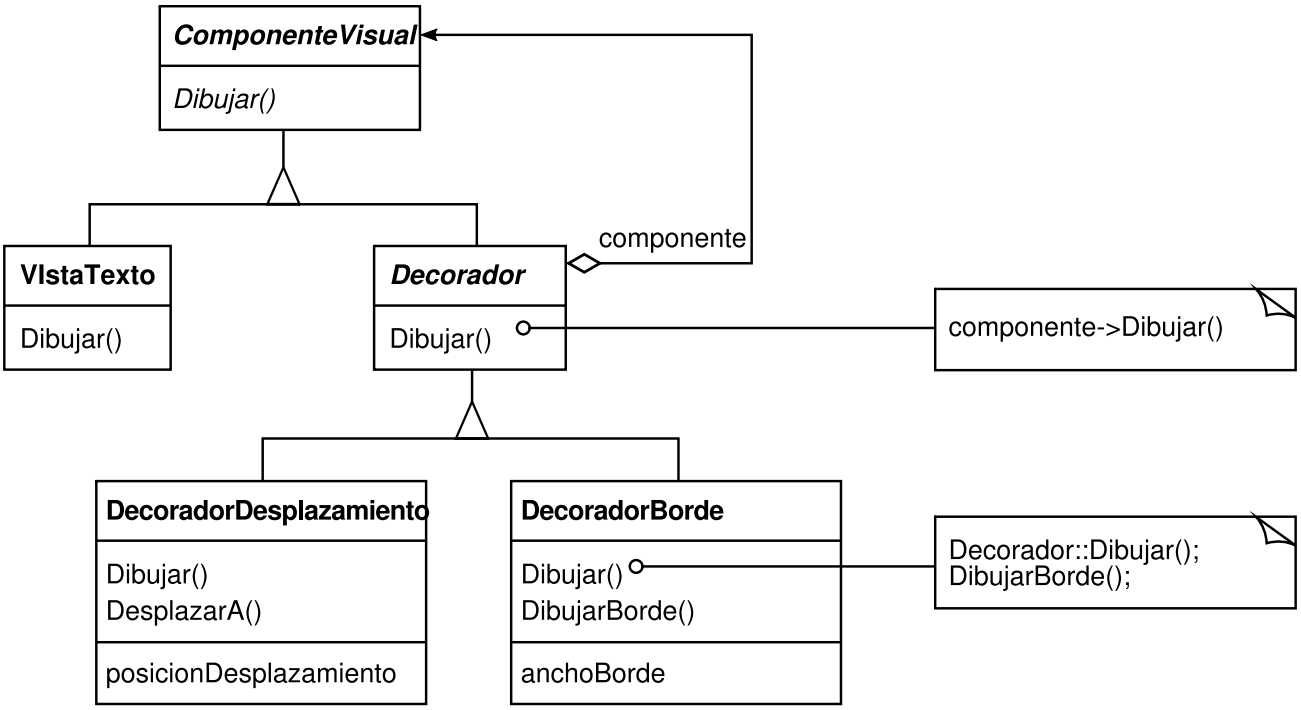
Un enfoque más flexible es encerrar el componente en otro objeto que añada el borde. Al objeto confinante se le denomina **decorador**. El decorador se ajusta a la interfaz del componente que decora de manera que su presencia es transparente a sus clientes. El decorador reenvía las peticiones al componente y puede realizar acciones adicionales (tales como dibujar un borde) antes o después del reenvío. Dicha transparencia permite anidar decoradores recursivamente, permitiendo así un número ilimitado de responsabilidades añadidas.



Por ejemplo, supongamos un objeto VistaTexto que muestra texto en una ventana. VistaTexto no tiene barras de desplazamiento de manera predeterminada, ya que puede que no sean siempre necesarias. Cuando las necesitemos, podemos usar un DecoradorDesplazamiento para añadirlas. Supongamos que queremos añadir un borde negro, grueso, alrededor de VistaTexto. Podemos usar un DecoradorBorde para añadir también el borde. Basta con componer los decoradores con VistaTexto para producir el resultado deseado.

El siguiente diagrama de objetos muestra cómo componer un objeto VistaTexto con objetos DecoradorBorde y DecoradorDesplazamiento para producir una vista de texto con borde y desplazamiento:

Las clases DecoradorDesplazamiento y DecoradorBorde son subclases de Decorador, una clase abstracta para componentes visuales que decoran otros componentes visuales.



Componente Visual es la clase abstracta de los objetos visuales. Define su interfaz para dibujarse y para el manejo de eventos. Nótese cómo la clase Decorador simplemente redirige las peticiones para que se dibuje su componente, y cómo las subclases de Decorador pueden extender dicha operación.

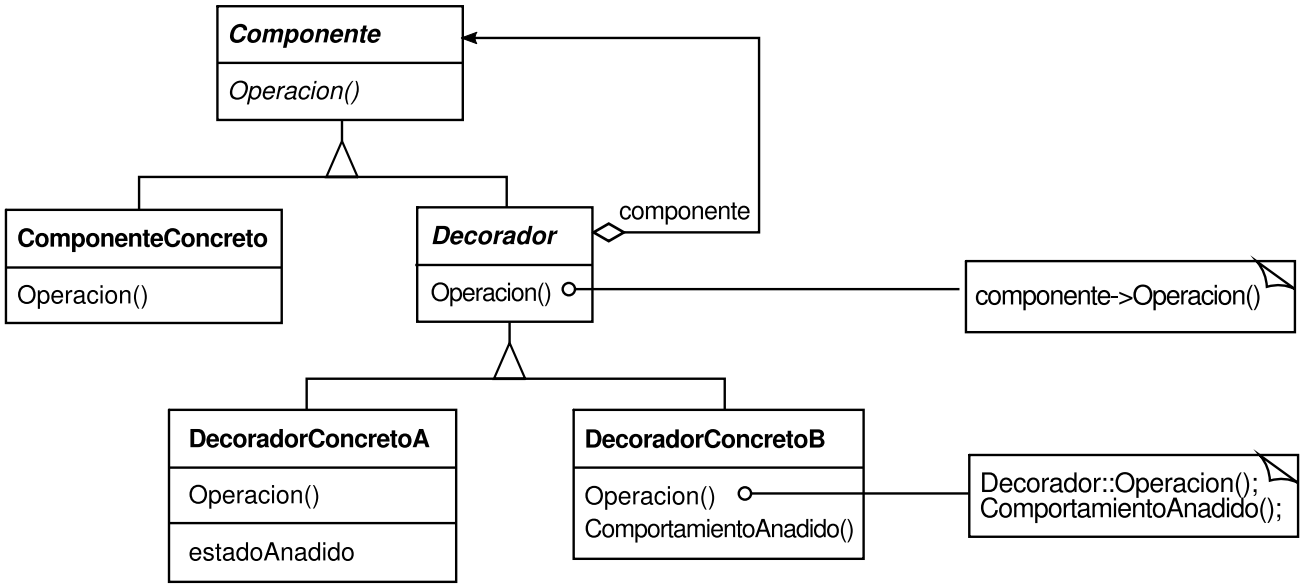
Las subclases de Decorador son libres de añadir operaciones para determinadas funcionalidades. Por ejemplo, la operación Desplazar A de DecoradorDesplazamiento permite que otros objetos desplacen la interfaz si saben que la interfaz incluye un objeto DecoradorDesplazamiento. Lo importante de este patrón es que permite que los decoradores aparezcan en cualquier lugar en el que pueda ir un Componente Visual. De esa forma los clientes generalmente no pueden distinguir entre un componente decorado y otro que no lo está, por lo que no dependen en absoluto de la decoración.

APLICABILIDAD

Use el Decorador

- para añadir objetos individuales de forma dinámica y transparente, es decir, sin afectar a otros objetos.
- para responsabilidades que pueden ser retiradas.
- cuando la extensión mediante la herencia no es viable. A veces es posible tener un gran número de extensiones independientes, produciéndose una explosión de subclases para permitir todas las combinaciones. O puede ser que una definición de una clase esté oculta o que no esté disponible para ser heredada.

ESTRUCTURA



PARTICIPANTES

- **Componente** (ComponenteVisual)
  - define la interfaz para objetos a los que se puede añadir responsabilidades dinámicamente.
- **ComponenteConcreto** (VistaTexto)
  - define un objeto al que se pueden añadir responsabilidades adicionales.
- **Decorador**
  - mantiene una referencia a un objeto Componente y define una interfaz que se ajusta a la interfaz del Componente.
- **DecoradorConcreto** (DecoradorBorde, DecoradorDesplazamiento)
  - añade responsabilidades al componente.

COLABORACIONES

El Decorador redirige peticiones a su objeto Componente. Opcionalmente puede realizar operaciones adicionales antes y después de reenviar la petición.

CONSECUENCIAS

El patrón Decorador tiene al menos dos ventajas y dos inconvenientes fundamentales:

1. *Más flexibilidad que la herencia estática*. El patrón Decorador proporciona una manera más flexible de añadir responsabilidades a los objetos que la que podía obtenerse a través de la herencia (múltiple) estática. Con los decoradores se

pueden añadir y eliminar responsabilidades en tiempo de ejecución simplemente poniéndolas y quitándolas. Por el contrario, la herencia requiere crear una nueva clase para cada responsabilidad adicional (como VistaTextoDesplazableConBorde o VistaTextoConBorde). Esto da lugar a muchas clases diferentes e incrementa la complejidad de un sistema. Por otro lado, proporcionar diferentes clases Decorador para una determinada clase Componente permite mezclar responsabilidades.

Los Decoradores también facilitan añadir una propiedad dos veces. Por ejemplo, para dar un borde doble a VistaTexto, basta con añadir dos objetos DecoradorBorde. Heredar dos veces de una clase Borde resulta, cuando menos, propenso a errores.

2. *Evita clases cargadas de funciones en la parte de arriba de la jerarquía*. El Decorador ofrece un enfoque para añadir responsabilidades que consiste en pagar sólo por aquello que se necesita. En vez de tratar de permitir todas las funcionalidades inimaginables en una clase compleja y adaptable, podemos definir primero una clase simple y añadir luego funcionalidad incrementalmente con objetos Decorador. La funcionalidad puede obtenerse componiendo partes simples. Como resultado, una aplicación no necesita pagar por características que no usa. También resulta fácil definir nuevos tipos de Decoradores independientemente de las clases de objetos de las que hereden, incluso para extensiones que no hubieran sido previstas. Extender una clase compleja tiende a exponer detalles no relacionados con las responsabilidades que estamos añadiendo.
3. *Un decorador y su componente no son idénticos*. Un decorador se comporta como un revestimiento transparente. Pero desde el punto de vista de la identidad de un objeto, un componente decorado no es idéntico al componente en sí. Por tanto, no deberíamos apoyarnos en la identidad de objetos cuando estamos usando decoradores.
4. *Muchos objetos pequeños*. Un diseño que usa el patrón Decorator suele dar como resultado sistemas formados por muchos objetos pequeños muy parecidos. Los objetos sólo se diferencian en la forma en que están interconectados, y no en su clase o en el valor de sus variables. Aunque dichos sistemas son fáciles de adaptar por parte de quienes los comprenden bien, pueden ser difíciles de aprender y de depurar.

IMPLEMENTACIÓN

Hay que tener en cuenta varias cuestiones al aplicar el patrón Decorator:

1. *Concordancia de interfaces*. La interfaz de un objeto decorador debe ajustarse a la interfaz del componente que decora. Las clases DecoradorConcreto deben

por tanto heredar de una clase común (al menos en C++).

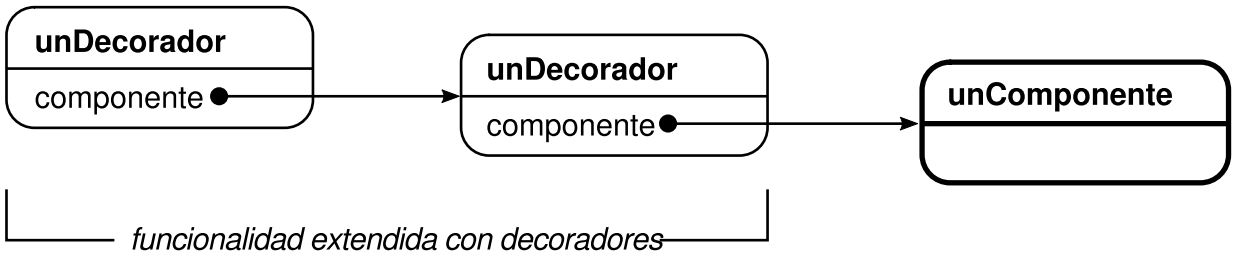
2. *Omisión de la clase abstracta Decorador*. No hay necesidad de definir una clase abstracta Decorador cuando sólo necesitamos añadir una responsabilidad. Eso es lo que suele ocurrir cuando estamos tratando con una jerarquía de clases existente y no diseñando una nueva. En ese caso, podemos obtener la responsabilidad del Decorador reenviando peticiones al componente en el DecoradorConcreto.

3. *Mantener ligeras las clases Componente*. Para garantizar una interfaz compatible, los componentes y los decoradores deben descender de una clase Componente común. Es importante que esta clase común se mantenga ligera; es decir, debería centrarse en definir una interfaz, no en guardar datos. La definición de cómo se representan los datos debería delegarse en las subclases; de no ser así, la complejidad de la clase Componente puede hacer que los decoradores sean demasiado pesados como para usar una gran número de ellos. Poner mucha funcionalidad en el Componente también incrementa la probabilidad de que las subclases concretas estén pagando por características que no necesitan.

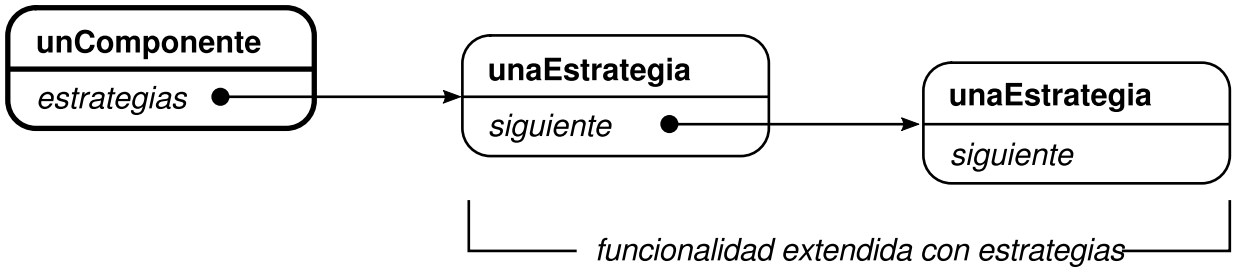
4. *Cambiar la piel de un objeto en vez de sus tripas*. Podemos pensar en un decorador como un revestimiento de un objeto que cambia su comportamiento. Una alternativa es cambiar las interioridades del objeto. El patrón Strategy (289) es un buen ejemplo de patrón para cambiar las tripas. Las estrategias son una mejor elección en aquellas situaciones en las que la clase Componente es intrínsecamente pesada, lo que hace que el patrón Decorator sea demasiado costoso de aplicar. En el patrón Strategy, el componente delega parte de su responsabilidad en un objeto estrategia aparte. El patrón Strategy nos permite alterar o extender la funcionalidad del componente cambiando el objeto estrategia.

   Por ejemplo, podemos permitir diferentes estilos de bordes haciendo que el componente delegue el dibujado del borde en un objeto Borde aparte. El objeto Borde es un objeto Estrategia que encapsula un algoritmo para dibujar bordes. Al ampliar el número de estrategias de una a una lista ilimitada conseguimos el mismo efecto que anidando recursivamente decoradores. Por ejemplo, en MacApp 3.0 [App89] y Bedrock [Sym93a] los componentes gráficos (denominados “vistas”) mantienen una lista de objetos “adorno” que pueden añadir adornos adicionales, como bordes a un componente vista. Si una vista tiene algunos adornos, les da una posibilidad de dibujar adornos adicionales. MacApp y Bedrock deben usar este enfoque debido a que la clase View es pesada, por lo que sería demasiado costoso usarla sólo para añadir un borde.

Puesto que el patrón Decorator sólo cambia la parte exterior de un objeto, el componente no tiene que saber nada de sus decoradores; es decir, los decoradores son transparentes para el componente:



Con estrategias, el componente conoce sus posibles extensiones. Por tanto tiene que referenciar y mantener las estrategias correspondientes:



El enfoque basado en estrategias puede requerir modificar el componente para permitir nuevas extensiones. Por otro lado, una estrategia puede tener su propia interfaz especializada, mientras que la interfaz de un decorador debe ajustarse a la del componente. Una estrategia para dibujar un borde, por ejemplo, sólo necesita definir la interfaz para dibujar el borde (DibujarBorde, ObtenerAncho, etc.), lo que significa que la estrategia puede ser ligera incluso aunque la clase componente sea pesada.

MacApp y Bedrock usan este enfoque para algo más que simplemente adornar vistas. También la usan para aumentar el comportamiento de manejo de eventos de los objetos. En ambos sistemas, una vista mantiene una lista de objetos de "comportamiento" que pueden modificare interceptar eventos. La vista da la posibilidad de manejar el evento a cada uno de los objetos de comportamiento registrados antes que a los no registrados. Podemos decorar una vista para que admita el manejo de eventos, por ejemplo, registrando un objeto comportamiento que intercepte y maneje los eventos de teclado.

CÓDIGO DE EJEMPLO

El código siguiente muestra cómo implementar decoradores de interfaz de usuario en C++. Supondremos que hay una clase Componente llamada ComponenteVisual.

```
class ComponenteVisual {
public:
     ComponenteVisual();
```

```
     virtual void Dibujar();
     virtual void CambiarTamano();
     // …
};
```

Definiremos una subclase de ComponenteVisual llamada Decorador, de la cual heredaremos para obtener diferentes decoraciones.

```
class Decorador : public ComponenteVisual {
public:
     Decorator(ComponenteVisual*);

     virtual void Dibujar();
     virtual void CambiarTamano();
     // …
private:
     ComponenteVisual* _componente;
};
```

Decorador decora el ComponenteVisual referenciado por la variable de instancia _componente, la cual es inicializada en el constructor. Para cada operación de la interfaz ComponenteVisual, Decorador define una implementación predeterminada que pasa la petición a _componente:

```
void Decorador::Dibujar () {
     _componente->Dibujar();
}

void Decorador::CambiarTamano () {
     _componente->CambiarTamano();
}
```

Las subclases de Decorador definen decoraciones concretas. Por ejemplo, la clase DecoradorBorde añade un borde a su componente. DecoradorBorde es una subclase de Decorador que redefine la operación Dibujar para dibujar el borde. DecoradorBorde también define una operación privada auxiliar DibujarBorde que se encarga de dibujarlo. La subclase hereda de Decorador la implementación de todas las otras operaciones.

```
class DecoradorBorde : public Decorator {
public:
     DecoradorBorde(ComponenteVisual*, int anchoBorde);

     virtual void Dibujar();
private:
     void DibujarBorde(int);
private:
     int _ancho;
};

void DecoradorBorde::Dibujar () {
     Decorator::Dibujar();
     DibujarBorde(_ancho);
}
```

Aquí vendría una implementación parecida para DecoradorDesplazamiento y DecoradorSombra, que añadirían capacidades de desplazamiento y sombra a un componente visual.

Ahora podemos combinar instancias de estas clases para proporcionar diferentes decoraciones. El código siguiente ilustra cómo podemos usar decoradores para crear una VistaTexto desplazable y con borde.

En primer lugar, necesitamos un modo de poner un objeto visual en un objeto ventana. Supondremos que nuestra clase Ventana proporciona una operación EstablecerContenido para este propósito:

```
void Ventana::EstablecerContenido (ComponenteVisual* contenido) {
     // …
}
```

Ahora podemos crear la vista de texto y una ventana donde ponerla:

```
Ventana* ventana = new Ventana;
VistaTexto* VistaTexto = new VistaTexto;
```

VistaTexto es un ComponenteVisual, lo que nos permite ponerla en la ventana:

```
ventana->EstablecerContenido(vistaTexto);
```

Pero queremos una VistaTexto con borde y que se pueda desplazar, por lo que hemos de decorarla de manera apropiada antes de ponerla en la ventana.

```
ventana->EstablecerContenido(
     new DecoradorBorde(
          new DecoradorDesplazamiento(vistaTexto), 1
     )
);
```

Dado que Ventana accede a su contenido a través de la interfaz ComponenteVisual, no se percata de la presencia del decorador. Nosotros, como clientes, todavía podemos utilizar la vista de texto si tenemos que interactuar con ella directamente, por ejemplo cuando necesitamos invocar operaciones que no son parte de la interfaz de ComponenteVisual. Los clientes que se basan en la identidad del componente también deberían referirse a ella directamente.

USOS CONOCIDOS

Muchos toolkits de interfaces de usuario orientados a objetos usan decoradores para añadir adornos gráficos a los útiles[38]. Algunos ejemplos son Interviews [LVC98, LCI+92], ET++ [WGM88] y la biblioteca de clases ObjectWorksVSmalltalk [Par90]. Aplicaciones más exóticas del patrón Decorator son el DebuggingGlyph de Interviews y el PassivityWrapper de ParcPlace Smalltalk. Un DebuggingGlyph

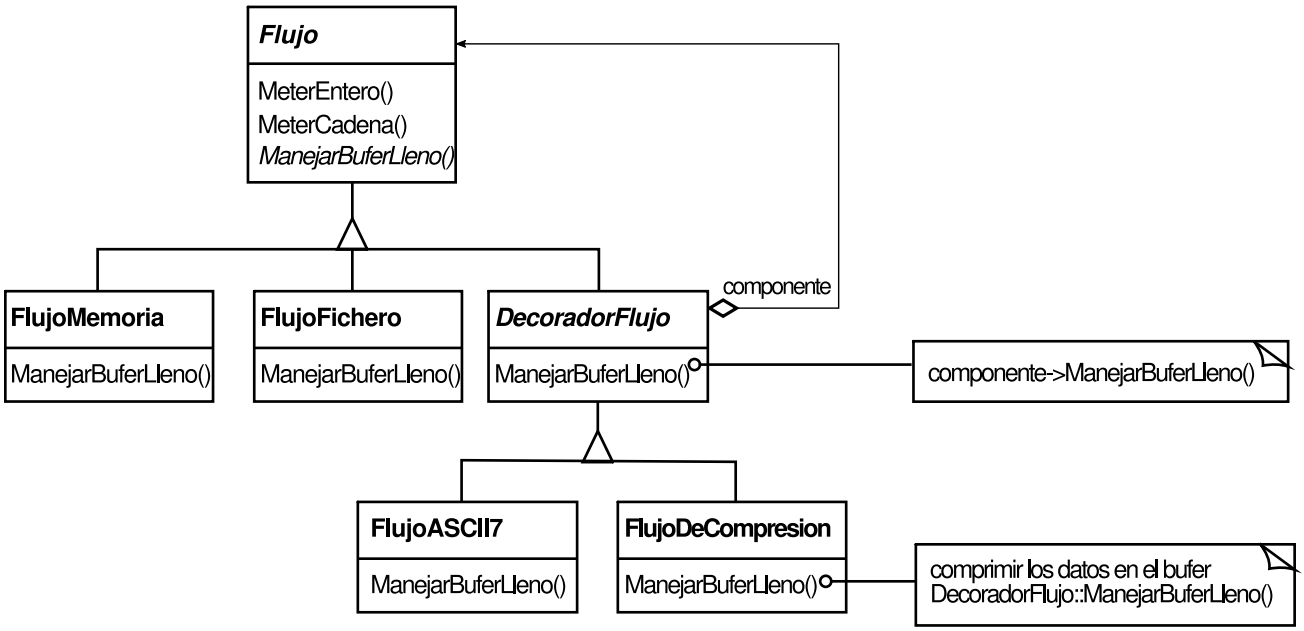
muestra información de depuración antes y después de reenviar una petición a su componente para que se ubique en pantalla. Esta información de traza puede usarse para analizar y depurar el comportamiento de cómo se disponen los objetos en una composición compleja. El PassivityWrapper puede activar o desactivar las interacciones del usuario con el componente.

Pero el patrón Decorator no se limita en absoluto a las interfaces gráficas de usuario, como demuestra el siguiente ejemplo (basado en las clases de flujos de ET++ [WGM88]).

Los flujos son una abstracción fundamental en la mayoría de mecanismos de E/S. Un flujo puede proporcionar una interfaz para convertir objetos en una secuencia de bytes o caracteres. Eso nos permite transcribir un objeto a un fichero o a una cadena en memoria para recuperarlo posteriormente. Un modo sencillo de llevar esto a cabo es definiendo una clase abstracta Flujo con subclases como FlujoMemoria y FlujoFichero. Pero supongamos que también queremos poder hacer lo siguiente:

- Comprimir el flujo de datos usando diferentes algoritmos de compresión (run-length encoding, Lempel-Ziv, etc.).
- Reducir el flujo de datos a caracteres ASCII de 7 bits, de manera que puedan transmitirse sobre un canal de comunicación ASCII.

El patrón Decorador nos ofrece un modo elegante de añadir estas responsabilidades a los flujos. El siguiente diagrama muestra una solución al problema:



La clase abstracta Flujo mantiene un búfer interno y proporciona operaciones para almacenar datos en el flujo (MeterEntero, MeterCadena). Cuando se llene el búfer, Flujo llamará a la operación ManejarBuferLleno, que lleva a cabo la transferencia

real de datos. La versión FlujoFichero de esta operación la redefine para transferir el búfer a un fichero.

La clase fundamental aquí es DecoradorFlujo, que mantiene una referencia a un componente flujo y le reenvía peticiones. Las subclases de DecoradorFlujo redefinen ManejarBuferLleno y llevan a cabo acciones adicionales llamando a la operación ManejarBuferLleno de DecoradorFlujo.

Por ejemplo, la subclase FlujoDeCompresion comprime los datos, y FlujoASC117 convierte los datos en ASCII de 7 bits. Ahora, para crear un FlujoFichero que comprima sus datos y convierta los datos binarios comprimidos en ASCII de 7 bits, decoramos un FlujoFichero con un FlujoDeCompresion y un FlujoASCII7:

```
Flujo* unFlujo = new FlujoCompresion(
     new FlujoASCII7(
          new FlujoFichero("unNombreDeFichero")
     )
);
unFlujo->MeterEntero(12);
unFlujo->MeterCadena ("unaCadena");
```

PATRONES RELACIONADOS

Adapter (131): un decorador se diferencia de un adaptador en que el decorador sólo cambia las responsabilidades de un objeto, no su interfaz, mientras que un adaptador le da a un objeto una interfaz completamente nueva.

Composite (151): podemos ver a un decorador como un compuesto degenerado que sólo tiene un componente. No obstante, un decorador añade responsabilidades adicionales —no está pensado para la agregación de objetos—.

Strategy (289): un decorador permite cambiar el exterior de un objeto; una estrategia permite cambiar sus tripas. Son dos formas alternativas de modificar un objeto.